COSC 2633/2637
Big Data Processing

Lecture 4

MapReduce Algorithm Design Patterns via Co-occurrence Matrix

RMIT UNIVERSITY

Part 1
Co-occurrence Matrix

MapReduce - Pros and Cons

Pros – Simplicity

- Programmer needs
  - Prepare the input data
  - Implement the mapper
  - Implement the reducer
  - Implement the combiner (optional)
  - Implement the partitioner (optional)

MapReduce - Pros and Cons

Pros – Simplicity

- All other aspects of execution are handled transparently by the execution framework on clusters
  - Handles scheduling
  - Handles "data distribution"
  - Handles synchronization
  - Handles errors and faults

MapReduce - Pros and Cons

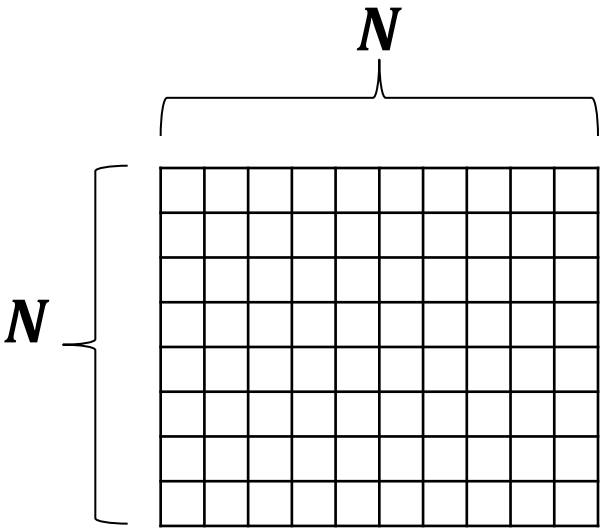
Cons – inflexibility

- Any algorithm that a programmer develop must be expressed
  - In a small number of rigidly-defined components
  - Fit them together in very specific way

No straightforward to recast an algorithm into this programming model

Co-occurrence Matrix

Given a text collection (like a large number of web pages), we are interested in term co-occurrence matrix



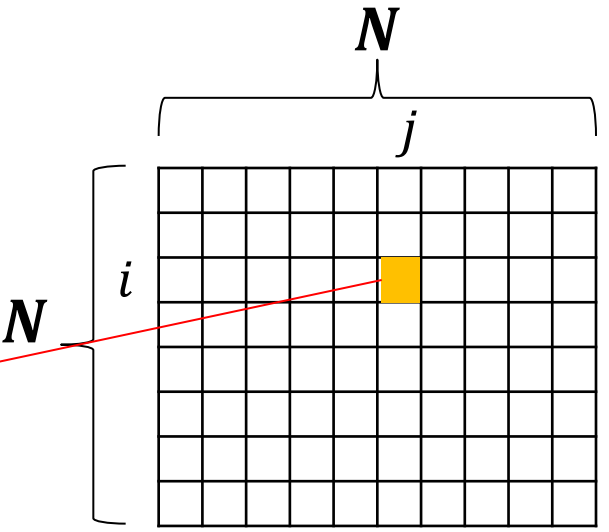
N : the number of terms in a vocabulary

$M = N \times N$ matrix

M_{ij} : An element in matrix M, the value of M_{ij} is the number of times *i* and *j* co-occur in some context (for concreteness, let's say context = sentence)

Co-occurrence Matrix

Given a text collection (like a large number of web pages), we are interested in term co-occurrence matrix

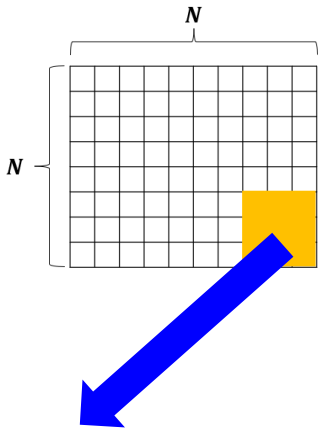


N : the number of terms in a vocabulary

$M = N \times N$ matrix

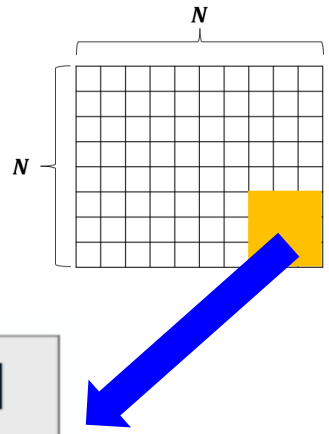
M_{ij} : An element in matrix M, the value of M_{ij} is the number of times *i* and *j* co-occur in some context (for concreteness, let's say context = sentence)

Co-occurrence Matrix - Example



Term co-occurrence matrix - example

Co-occurrence Matrix - Example



Term co-occurrence matrix - example

	get	see	use	hear	eat	kill
knife	51	20	84	0	3	0
cat	52	58	4	4	6	26
dog	**115**	**89**	**10**	**42**	**33**	**17**
boat	59	39	23	4	0	0
cup	98	14	6	2	1	0
pig	12	17	3	2	9	27
berry	11	2	2	0	18	0

"dog" and "see" co-occur in 89 sentences

Co-occurrence Matrix

What co-occurrence matrix can be used for?

- Distributional profiles as a way of measuring semantic distance
- Semantic distance useful for many language processing tasks

	get	see	use	hear	eat	kill
knife	51	20	84	0	3	0
cat	52	58	4	4	6	26
dog	**115**	**89**	**10**	**42**	**33**	**17**
boat	59	39	23	4	0	0
cup	98	14	6	2	1	0
pig	12	17	3	2	9	27
berry	11	2	2	0	18	0

Co-occurrence Matrix

What co-occurrence matrix can be used for?

- Distributional profiles as a way of measuring semantic distance
- Semantic distance useful for many language processing tasks

	get	see	use	hear	eat	kill
knife	51	20	84	0	3	0
cat	52	58	4	4	6	26
dog	**115**	**89**	**10**	**42**	**33**	**17**
boat	59	39	23	4	0	0
cup	98	14	6	2	1	0
pig	12	17	3	2	9	27
berry	11	2	2	0	18	0



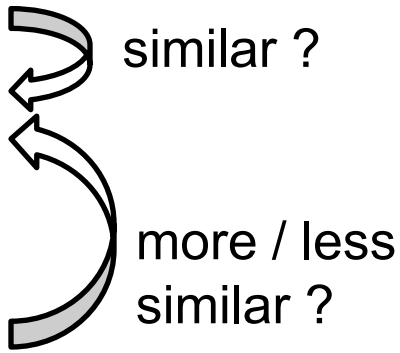
similar ?

Co-occurrence Matrix

What co-occurrence matrix can be used for?

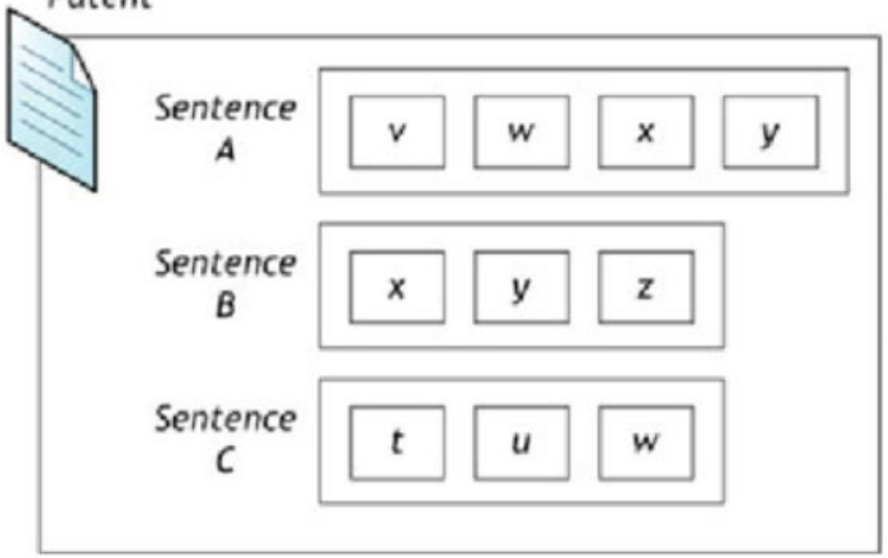
- Distributional profiles as a way of measuring semantic distance
- Semantic distance useful for many language processing tasks

	get	see	use	hear	eat	kill
knife	51	20	84	0	3	0
cat	52	58	4	4	6	26
dog	**115**	**89**	**10**	**42**	**33**	**17**
boat	59	39	23	4	0	0
cup	98	14	6	2	1	0
pig	12	17	3	2	9	27
berry	11	2	2	0	18	0



Co-occurrence Matrix

From Documents to Term Co-occurrence Matrix

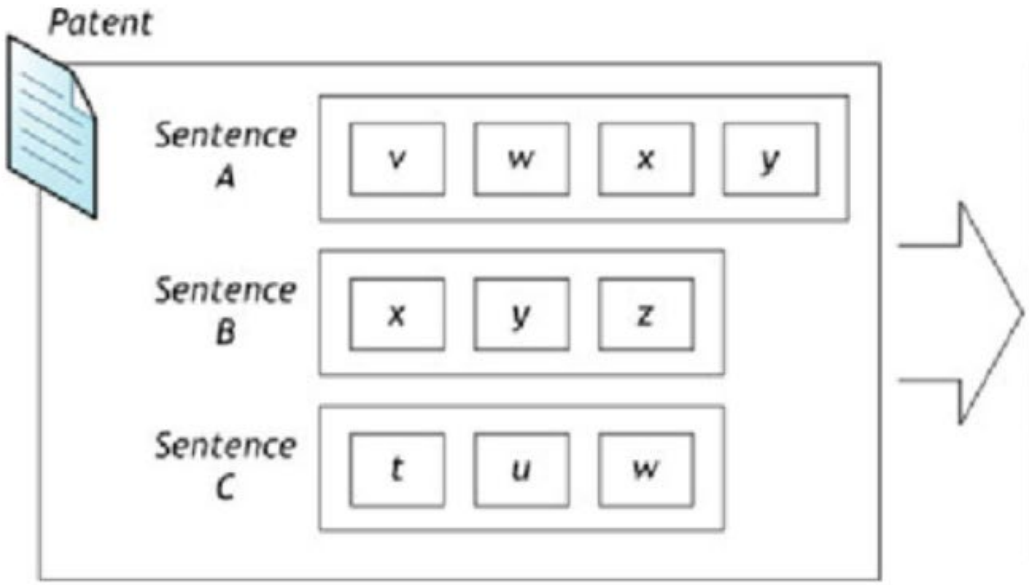


Co-occurrence matrix

	v	w	x	y	z	t	u
v	0	1	1	1	0	0	0
w	1	0	1	1	0	1	1
x	1	1	0	2	1	0	0
y	1	1	2	0	1	0	0
z	0	0	1	1	0	0	0
t	0	1	0	0	0	0	1
u	0	1	0	0	0	1	0

Co-occurrence Matrix

From Documents to Term Co-occurrence Matrix



Co-occurrence matrix

	v	w	x	y	z	t	u
v	0	1	1	1	0	0	0
w	1	0	1	1	0	1	1
x	1						0
y	1						0
z	0						0
t	0	1	0	0	0	0	1
u	0	1	0	0	0	1	0

7x7=49

Simple if the entire matrix fits into memory

Very slow if the matrix is too big to fit in memory

10,000 items in a vocabulary

Co-occurrence Matrix has 100,000,000 elements

https://en.wikipedia.org/wiki/List_of_dictionaries_by_number_of_words

Compute Co-occurrence Matrix

- **Two MapReduce Algorithms**
  - Pairs Approach
  - Stripes Approach

Compute Co-occurrence Matrix

Pairs Approach

doc d (i.e., a line of a document in practice)

mapper.py

Input: $doc\ d$

for each term $w \in doc\ d$ do

    for each term $u \in Neighbour(w)$ do

        emit(pair(w, v), count 1)

reducer.py

Input: pair p, count $[c1, c2, \dots]$

$s \leftarrow 0$

for each $c \in count[c1, c2, \dots]$ do

    $s \leftarrow s + c$

emit(pair p, count s)

You can specify a file name. The file is split into blocks. Each map task processes one block. The content of the block is fed to mapper.py line by line.

You can specify a folder name. All files are split into blocks and processed in the same way.

Compute Co-occurrence Matrix

Pairs Approach

mapper.py

Input: $doc\ d$

for each term $w \in doc\ d$ do

    for each term $u \in Neighbour(w)$ do

        emit(pair(w, v), count 1)

doc d (i.e., a line of a document in practice)

The neighbor of a word can either be defined within a sliding window or some other contextual unit such as a sentence.

Mapper emit for each (w, v)
Key: (w, v)
Value: 1

reducer.py

Input: pair p, count $[c1, c2, \dots]$

$s \leftarrow 0$

for each $c \in count[c1, c2, \dots]$ do

    $s \leftarrow s + c$

emit(pair p, count s)

The key is complex since it is a pair of terms

For example: ((cat, see), 1)

Compute Co-occurrence Matrix

Pairs Approach

MapReduce execution framework guarantees all values counts[c_1, c_2,…] associated with the same key pair p are brought together in the same reducer.

mapper.py

Input: $doc\ d$
for each term $w \in doc\ d$ do
    for each term $u \in Neighbour(w)$ do
        emit(pair(w, v), count 1)

reducer.py

Input: pair p, count $[c1, c2, \dots]$
$s \leftarrow 0$
for each $c \in count[c1, c2, \dots]$ do
    $s \leftarrow s + c$
emit(pair p, count s)

Reducer simply sums up all the values associated with the same key pair p

Emit the key pair p and the sum

For example: ((cat, see), 58)

	get	see	use	hear	eat	kill
knife	51	20	84	0	3	0
cat	52	58	4	4	6	26
dog	115	89	[illegible]	42	33	17
boat	59	39	23	4	0	0
cup	98	14	6	2	1	0
pig	12	17	3	2	9	27
berry	11	2	2	0	18	0

MapReduce execution framework guarantees all values counts[c_1, c_2,…] associated with the same key pair p are brought together in the same reducer.

reducer.py

Input: pair p, count $[c1, c2, \ldots]$

$s \leftarrow 0$

for each $c \in count[c1, c2, \ldots]$ do

    $s \leftarrow s + c$

emit(pair p, count s)

Reducer simply sums up all the values associated with the same key pair p

Emit the key pair p and the sum

For example: ((cat, see), 58)

Compute Co-occurrence Matrix

- **Two MapReduce Algorithms**
  - Pairs Approach
  - Stripes Approach

Compute Co-occurrence Matrix

Stripes Approach

doc d (i.e., a line of a document in practice)

mapper.py

Input: $doc\ d$

for each term $w \in doc\ d$ do

    $H \leftarrow$ a new list

    for each term $u \in Neighbour(w)$ do

        $H\{u\} \leftarrow H\{u\} + 1$

    emit(key w, stripe H)

You can specify a file name. The file is split into blocks. Each map task processes one block. The content of the block is fed to mapper.py line by line.

You can specify a folder name. All files are split into blocks and processed in the same way.

reducer.py

Input: key w, stripes $[H_1, H_2, \dots]$

$H_f \leftarrow$ a new list

for each stripe $H \in [H_1, H_2, \dots]$ do

    $H_f \leftarrow H_f + H$

emit(key w, stripe H_f)

Compute Co-occurrence Matrix

Stripes Approach

mapper.py

Input: $doc\ d$

for each term $w \in doc\ d$ do

$H \leftarrow$ a new list

for each term $u \in Neighbour(w)$ do

$H\{u\} \leftarrow H\{u\} + 1$

emit(key w, stripe H)

doc d (i.e., a line of a document in practice)

- a data structure H is created
- frequency of each neighbor of w is counted and maintained in H

emit key-value

- key - w (a single word not a word pair)
- value – the stripe H associated with w

$(cat, \{see, 1\}, \{get, 1\})$

key

stripe H where

$H\{see\} = 1$

$H\{get\} = 1$

reducer.py

Input: key w, stripes $[H_1, H_2, \dots]$

$H_f \leftarrow$ a new list

for each stripe $H \in [H_1, H_2, \dots]$ do

$H_f \leftarrow H_f + H$

emit(key w, stripe H_f)

RMIT Classification: Trusted

Compute Co-occurrence Matrix

Stripes Approach

({kill, 1}, {see, 1})

{see, 1}, {get, 1})

({kill, 1}, {eat, 1}, {use, 1})

(cat, H1, H2, H3, …)

MapReduce Execution Framework guarantees that all lists H with the same key w will be brought together in the reduce phase of processing.

mapper.py

Input: $doc\ d$

for each term $w \in doc\ d$ do

  $H \leftarrow$ a new list

  for each term $u \in Neighbour(w)$ do

    $H\{u\} \leftarrow H\{u\} + 1$

  emit(key w, stripe H)

Reducer performs an element-wise sum of list with the same key (i.e., word)

reducer.py

Input: key w, stripes $[H_1, H_2, \dots]$

$H_f \leftarrow$ a new list

for each stripe $H \in [H_1, H_2, \dots]$ do

  $H_f \leftarrow H_f + H$

emit(key w, stripe H_f)

RMIT Classification: Trusted

Compute Co-occurrence Matrix

Stripes Approach

({kill, 1}, {see, 1})

{see, 1}, {get, 1})

({kill, 1}, {eat, 1}, {use, 1})

(cat, H1, H2, H3, …)

MapReduce Execution Framework guarantees that all lists H with the same key w will be brought together in the reduce phase of processing.

mapper.py

Input: $doc\ d$

for each term $w \in doc\ d$ do

$H \leftarrow$ a new list

for each term $u \in Neighbour(w)$ do

$H\{u\} \leftarrow H\{u\} + 1$

emit(key w, stripe H)

Reducer performs an element-wise sum of list with the same key (i.e., word)

reducer.py

Input: key w, stripes $[H_1, H_2, \ldots]$

$H_f \leftarrow$ a new list

for each stripe $H \in [H_1, H_2, \ldots]$ do

$H_f \leftarrow H_f + H$

emit(key w, stripe H_f)

	get	see	use	hear	eat	kill
knife	51	20	84	0	3	0
cat	52	58	4	4	6	26
dog	**115**	**89**	**10**	**42**	**33**	**17**
boat	59	39	23	4	0	0
cup	98	14	6	2	1	0
pig	12	17	3	2	9	27
berry	11	2	2	0	18	0

RMIT Classification: Trusted

Compute Co-occurrence Matrix

Stripes Approach

({kill, 5}, {see, 4})

{see, 3}, {get, 5})

({kill, 12}, {eat, 3}, {use, 3})

(cat, H1, H2, H3, …)

MapReduce Execution Framework guarantees that all lists H with the same key w will be brought together in the reduce phase of processing.

mapper.py

Input: $doc\ d$

for each term $w \in doc\ d$ do

$H \leftarrow$ a new list

for each term $u \in Neighbour(w)$ do

$H\{u\} \leftarrow H\{u\} + 1$

emit(key w, stripe H)

Reducer performs an element-wise sum of list with the same key (i.e., word)

reducer.py

Input: key w, stripes $[H_1, H_2, \ldots]$

$H_f \leftarrow$ a new list

for each stripe $H \in [H_1, H_2, \ldots]$ do

$H_f \leftarrow H_f + H$

emit(key w, stripe H_f)

	get	see	use	hear	eat	kill
knife	51	20	84	0	3	0
cat	52	58	4	4	6	26
dog	115	89	10	42	33	17
boat	59	39	23	4	0	0
cup	98	14	6	2	1	0
pig	12	17	3	2	9	27
berry	11	2	2	0	18	0

End of Part 1

Part 2
Co-occurrence Matrix
Stripes vs Pairs

Pairs

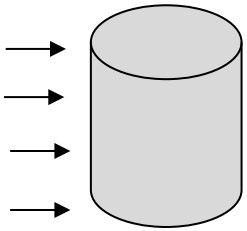
mapper.py

Input: $doc\ d$
for each term $w \in doc\ d$ do
    for each term $u \in Neighbour(w)$ do
        emit(pair(w, v), count 1)

reducer.py

Input: pair p, count $[c1, c2, \ldots]$
$s \leftarrow 0$
for each $c \in count[c1, c2, \ldots]$ do
    $s \leftarrow s + c$
emit(pair p, count s)

(("hadoop", "apache"): 1)
(("hadoop", "consists"): 1)
(("hadoop", "core"): 1)
(("hadoop", "storage"): 1)
….



…..
The core of Apache Hadoop consists of a storage part, known as Hadoop Distributed File System (HDFS), and a processing part which is a MapReduce programming model. Hadoop splits a file into large blocks and distributes them across nodes in a cluster. It then transfers packaged code into nodes to process the data in parallel.
…..

Pairs

mapper.py

Input: $doc\ d$
for each term $w \in doc\ d$ do
    for each term $u \in Neighbour(w)$ do
        emit(pair(w, v), count 1)

reducer.py

Input: pair p, count $[c1, c2, \ldots]$
$s \leftarrow 0$
for each $c \in count[c1, c2, \ldots]$ do
    $s \leftarrow s + c$
emit(pair p, count s)

Stripes

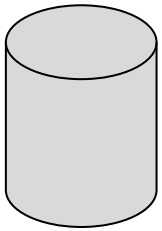
mapper.py

Input: $doc\ d$
for each term $w \in doc\ d$ do
    $H \leftarrow$ a new list
    for each term $u \in Neighbour(w)$ do
        $H\{u\} \leftarrow H\{u\} + 1$
    emit(key w, stripe H)

reducer.py

Input: key w, stripes $[H_1, H_2, \ldots]$
$H_f \leftarrow$ a new list
for each stripe $H \in [H_1, H_2, \ldots]$ do
    $H_f \leftarrow H_f + H$
emit(key w, stripe H_f)

(("hadoop", "apache"): 1)
(("hadoop", "consists"): 1)
(("hadoop", "core"): 1)
(("hadoop", "storage"): 1)
…



("hadoop", {{"apache": 1}, {"consists": 1}, {"core": 1}, {"storage": 1}})

…..
The core of Apache Hadoop consists of a storage part known as Hadoop Distributed File System (HDFS), and a processing part which is a MapReduce programming model. Hadoop splits a file into large blocks and distributes them across nodes in a cluster. It then transfers packaged code into nodes to process the data in parallel.
…..

Pairs

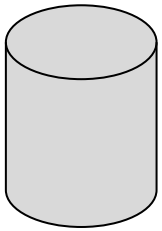
mapper.py

Input: $doc\ d$
for each term $w \in doc\ d$ do
    for each term $u \in Neighbour(w)$ do
        emit(pair(w, v), count 1)

reducer.py

Input: pair p, count $[c1, c2, \dots]$
$s \leftarrow 0$
for each $c \in count[c1, c2, \dots]$ do
    $s \leftarrow s + c$
emit(pair p, count s)

(("hadoop", "apache"): 1)
(("hadoop", "consists"): 1)
(("hadoop", "core"): 1)
(("hadoop", "storage"): 1)
…



Stripes

mapper.py

Input: $doc\ d$
for each term $w \in doc\ d$ do
    $H \leftarrow$ a new list
    for each term $u \in Neighbour(w)$ do
        $H\{u\} \leftarrow H\{u\} + 1$
    emit(key w, stripe H)

reducer.py

Input: key w, stripes $[H_1, H_2, \dots]$
$H_f \leftarrow$ a new list
for each stripe $H \in [H_1, H_2, \dots]$ do
    $H_f \leftarrow H_f + H$
emit(key w, stripe H_f)

("hadoop", {{"apache": 1}, {"consists": 1}, {"core": 1}, {"storage": 1}})

…..
The core of Apache Hadoop consists of a storage part, known as Hadoop Distributed File System (HDFS), and a processing part which is a MapReduce programming model. Hadoop splits a file into large blocks and distributes them across nodes in a cluster. It then transfers packaged code into nodes to process the data in parallel.
…..

RMIT Classification: Trusted

Pairs

((“hadoop”, “apache”): 1)
((“hadoop”, “consists”): 1)
((“hadoop”, “core”): 1)
((“hadoop”, “storage”): 1)

((“hadoop”, “known”): 1)
((“hadoop”, “distributed”): 1)
((“hadoop”, “file”): 1)
((“hadoop”, “part”): 1)

((“hadoop”, “model”): 1)
((“hadoop”, “programming”): 1)
((“hadoop”, “file”): 1)
((“hadoop”, “splits”): 1)

….

Stripes

(“hadoop”, {{“apache”: 1}, {“consists”: 1}, {“core”: 1}, {“storage”: 1}})

(“hadoop”, {{“known”: 1}, {“distributed”: 1}, {“file”: 1}, {“part”: 1}})

(“hadoop”, {{“model”: 1}, {“programming”: 1}, {“files”: 1}, {“splits”: 1}})

….

…..
The core of Apache Hadoop consists of a storage part, known as Hadoop Distributed File System (HDFS), and a processing part which is a MapReduce programming model. Hadoop splits a file into large blocks and distributes them across nodes in a cluster. It then transfers packaged code into nodes to process the data in parallel.
…..

Pairs

((“hadoop”, “apache”): 1)
((“hadoop”, “consists”): 1)
((“hadoop”, “core”): 1)
((“hadoop”, “storage”): 1)

((“hadoop”, “known”): 1)
((“hadoop”, “distributed”): 1)
((“hadoop”, “file”): 1)
((“hadoop”, “part”): 1)

((“hadoop”, “model”): 1)
((“hadoop”, “programming”): 1)
((“hadoop”, “file”): 1)
((“hadoop”, “splits”): 1)

….

Stripes

(“hadoop”, {{“apache”: 1}, {“consists”: 1}, {“core”: 1}, {“storage”: 1}})

(“hadoop”, {{“known”: 1}, {“distributed”: 1}, {“file”: 1}, {“part”: 1}})

(“hadoop”, {{“model”: 1}, {“programming”: 1}, {“files”: 1}, {“splits”: 1}})

….

…..
The core of Apache Hadoop consists of a storage part, known as Hadoop Distributed File System (HDFS), and a processing part which is a MapReduce programming model. Hadoop splits a file into large blocks and distributes them across nodes in a cluster. It then transfers packaged code into nodes to process the data in parallel.
…..

Pairs

mapper.py

Input: $doc\ d$

for each term $w \in doc\ d$ do

  for each term $u \in Neighbour(w)$ do

    emit(pair(w, v), count 1)

reducer.py

Input: pair p, count $[c1, c2, \ldots]$

$s \leftarrow 0$

for each $c \in count[c1, c2, \ldots]$ do

  $s \leftarrow s + c$

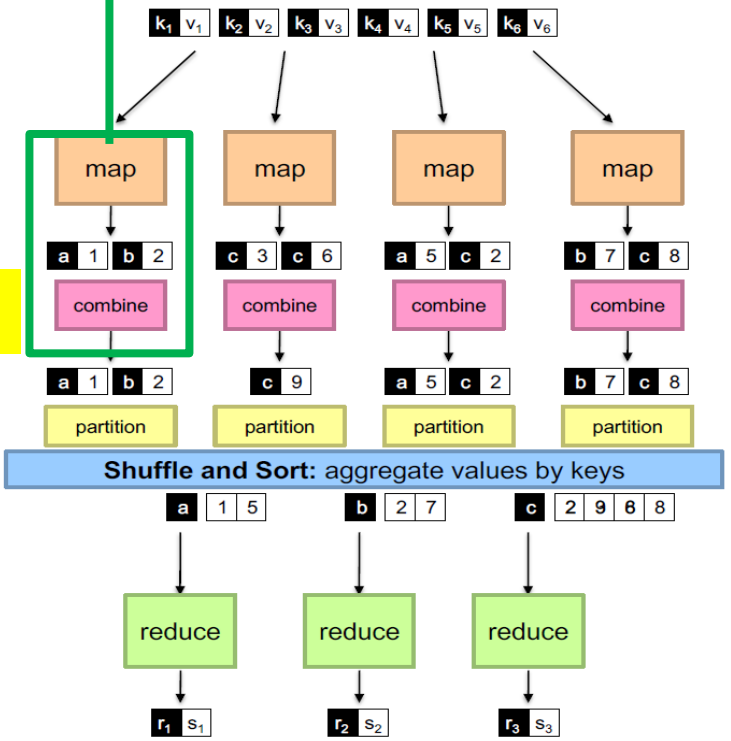
emit(pair p, count s)

(("hadoop", "apache"), 1)
(("hadoop", "consists"), 1)
(("hadoop", "core"), 1)
(("hadoop", "storage"), 1)

(("hadoop", "known"), 1)
(("hadoop", "distributed"), 1)
(("hadoop", "file"), 1)
(("hadoop", "part"), 1)

(("hadoop", "model"), 1)
(("hadoop", "programming"), 1)
(("hadoop", "file"), 1)
(("hadoop", "splits"), 1)

Combiner ?



Yes, operation is commutative and associative

Pairs

mapper.py

Input: $doc\ d$
for each term $w \in doc\ d$ do
    for each term $u \in Neighbour(w)$ do
        emit(pair(w, v), count 1)

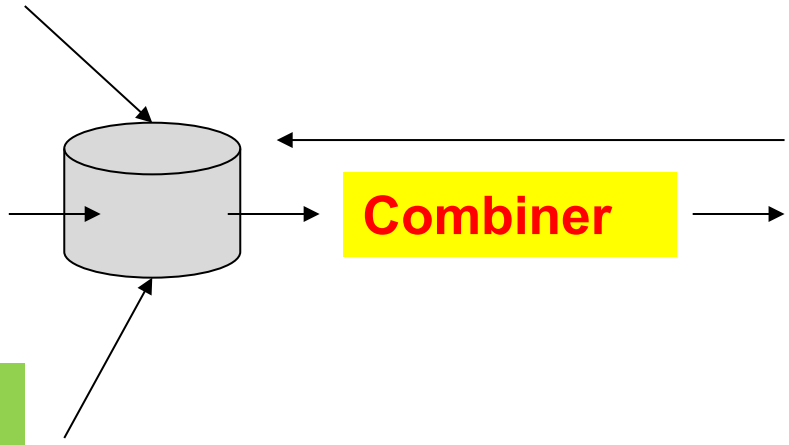
reducer.py

Input: pair p, count $[c1, c2, \ldots]$
$s \leftarrow 0$
for each $c \in count[c1, c2, \ldots]$ do
    $s \leftarrow s + c$
emit(pair p, count s)

(("hadoop", "apache"), 1)
(("hadoop", "consists"), 1)
(("hadoop", "core"), 1)
(("hadoop", "storage"), 1)

(("hadoop", "known"), 1)
(("hadoop", "distributed"), 1)
(("hadoop", "file"), 1)
(("hadoop", "part"), 1)

(("hadoop", "model"), 1)
(("hadoop", "programming"), 1)
(("hadoop", "file"), 1)
(("hadoop", "splits"), 1)



(("Hadoop", "apache"), 1)
(("Hadoop", "consists"), 1)
(("Hadoop", "core"), 1)
(("Hadoop", "storage"), 1)
(("Hadoop", "known"), 1)
(("Hadoop", "distributed"), 1)
(("Hadoop", "file"), 2)
(("Hadoop", "part"), 1)
(("Hadoop", "model"), 1)
(("Hadoop", "programming"), 1)
(("Hadoop", "splits"), 1)

Consider how to implement it ☺

Pairs

mapper.py

Input: $doc\ d$
for each term $w \in doc\ d$ do
    for each term $u \in Neighbour(w)$ do
        emit(pair(w, v), count 1)

reducer.py

Input: pair p, count $[c1, c2, \ldots]$
$s \leftarrow 0$
for each $c \in count[c1, c2, \ldots]$ do
    $s \leftarrow s + c$
emit(pair p, count s)

Consider how to implement it ☺
- By line
- Preserve state across lines

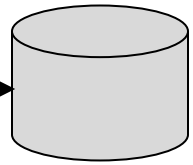
((“hadoop”, “apache”), 1)
((“hadoop”, “consists”), 1)
((“hadoop”, “core”), 1)
((“hadoop”, “storage”), 1)

((“hadoop”, “known”), 1)
((“hadoop”, “distributed”), 1)
((“hadoop”, “file”), 1)
((“hadoop”, “part”), 1)

((“hadoop”, “model”), 1)
((“hadoop”, “programming”), 1)
((“hadoop”, “file”), 1)
((“hadoop”, “splits”), 1)

In-mapper combining

((“Hadoop”, “apache”), 1)
((“Hadoop”, “consists”), 1)
((“Hadoop”, “core”), 1)
((“Hadoop”, “storage”), 1)
((“Hadoop”, “known”), 1)
((“Hadoop”, “distributed”), 1)
((“Hadoop”, “file”), 2)
((“Hadoop”, “part”), 1)
((“Hadoop”, “model”), 1)
((“Hadoop”, “programming”), 1)
((“Hadoop”, “splits”), 1)



RMIT Classification: Trusted

Stripes

mapper.py

Input: $doc\ d$
for each term $w \in doc\ d$ do
    $H \leftarrow$ a new list
    for each term $u \in Neighbour(w)$ do
        $H\{u\} \leftarrow H\{u\} + 1$
    emit(key w, stripe H)

reducer.py

Input: key w, stripes $[H_1, H_2, \dots]$
$H_f \leftarrow$ a new list
for each stripe $H \in [H_1, H_2, \dots]$ do
    $H_f \leftarrow H_f + H$
emit(key w, stripe H_f)

("hadoop", {{"apache": 1}, {"consists": 1}, {"core": 1}, {"storage": 1}})

("hadoop", {{"known": 1}, {"distributed": 1}, {"file": 1}, {"part": 1}})

("hadoop", {{"model": 1}, {"programming": 1}, {"files": 1}, {"splits": 1}})

….

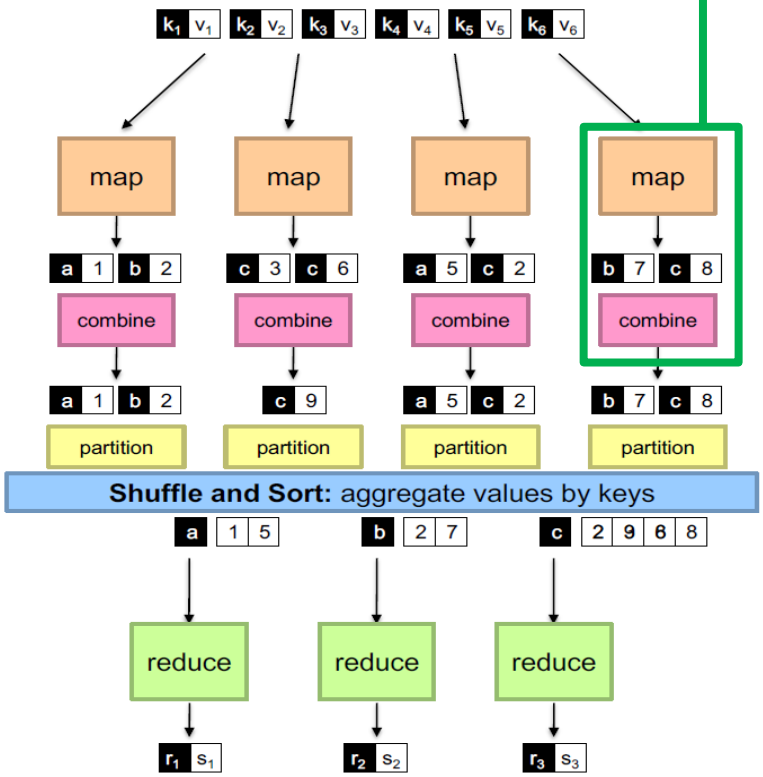
Stripes

mapper.py

Input: $doc\ d$
for each term $w \in doc\ d$ do
    $H \leftarrow$ a new list
    for each term $u \in Neighbour(w)$ do
        $H\{u\} \leftarrow H\{u\} + 1$
    emit(key w, stripe H)

reducer.py

Input: key w, stripes $[H_1, H_2, \dots]$
$H_f \leftarrow$ a new list
for each stripe $H \in [H_1, H_2, \dots]$ do
    $H_f \leftarrow H_f + H$
emit(key w, stripe H_f)

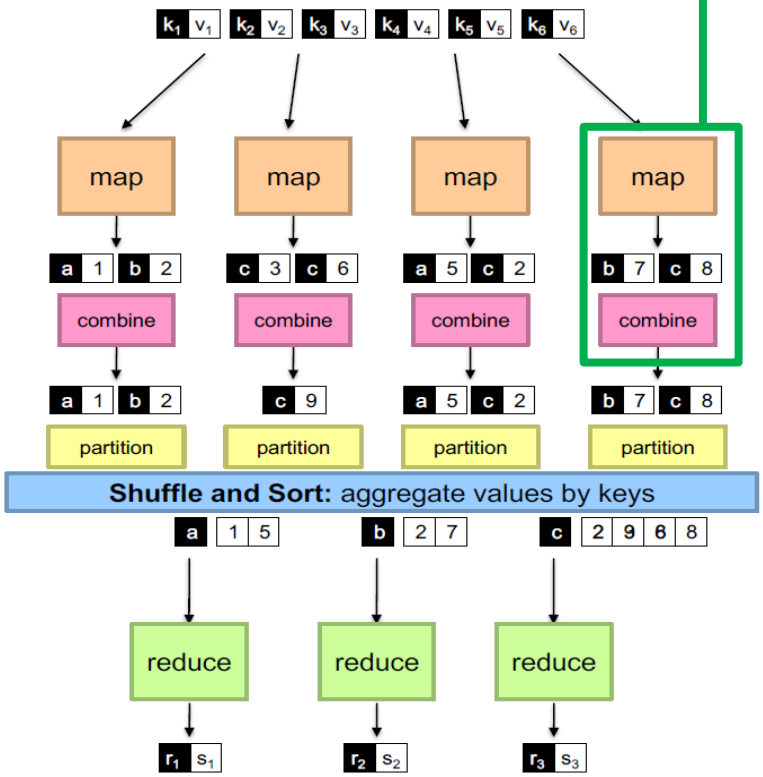


("hadoop", {{"apache": 1}, {"consists": 1}, {"core": 1}, {"storage": 1}})

("hadoop", {{"known": 1}, {"distributed": 1}, {"file": 1}, {"part": 1}})

("hadoop", {{"model": 1}, {"programming": 1}, {"files": 1}, {"splits": 1}})

….



Stripes

mapper.py

Input: $doc\ d$
for each term $w \in doc\ d$ do
    $H \leftarrow$ a new list
    for each term $u \in Neighbour(w)$ do
        $H\{u\} \leftarrow H\{u\} + 1$
    emit(key w, stripe H)

reducer.py

Input: key w, stripes $[H_1, H_2, \dots]$
$H_f \leftarrow$ a new list
for each stripe $H \in [H_1, H_2, \dots]$ do
    $H_f \leftarrow H_f + H$
emit(key w, stripe H_f)

Combiner ?

Yes, operation is commutative and associative

RMIT Classification: Trusted

("hadoop", {{"apache": 1}, {"consists": 1},
{"core": 1}, {"storage": 1}})

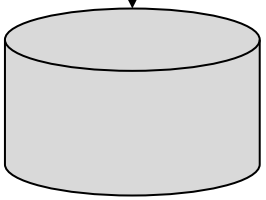
("hadoop", {{"known": 1}, {"distributed": 1},
{"file": 1}, {"part": 1}})

("hadoop", {{"model": 1}, {"programming": 1},
{"files": 1}, {"splits": 1}})

….

In-mapper Combining

("hadoop",
{{"apache", 1}, {"consists", 1},
{"core", 1}, {"storage" 1},
{"known", 1}, {"distributed", 1},
{"file", 2}, {"part" 1},
{"model", 1}, {"programming", 1},
{"splits" 1} })



Stripes

mapper.py

Input: $doc\ d$
for each term $w \in doc\ d$ do
  $H \leftarrow$ a new list
  for each term $u \in Neighbour(w)$ do
    $H\{u\} \leftarrow H\{u\} + 1$
  emit(key w, strip H)

reducer.py

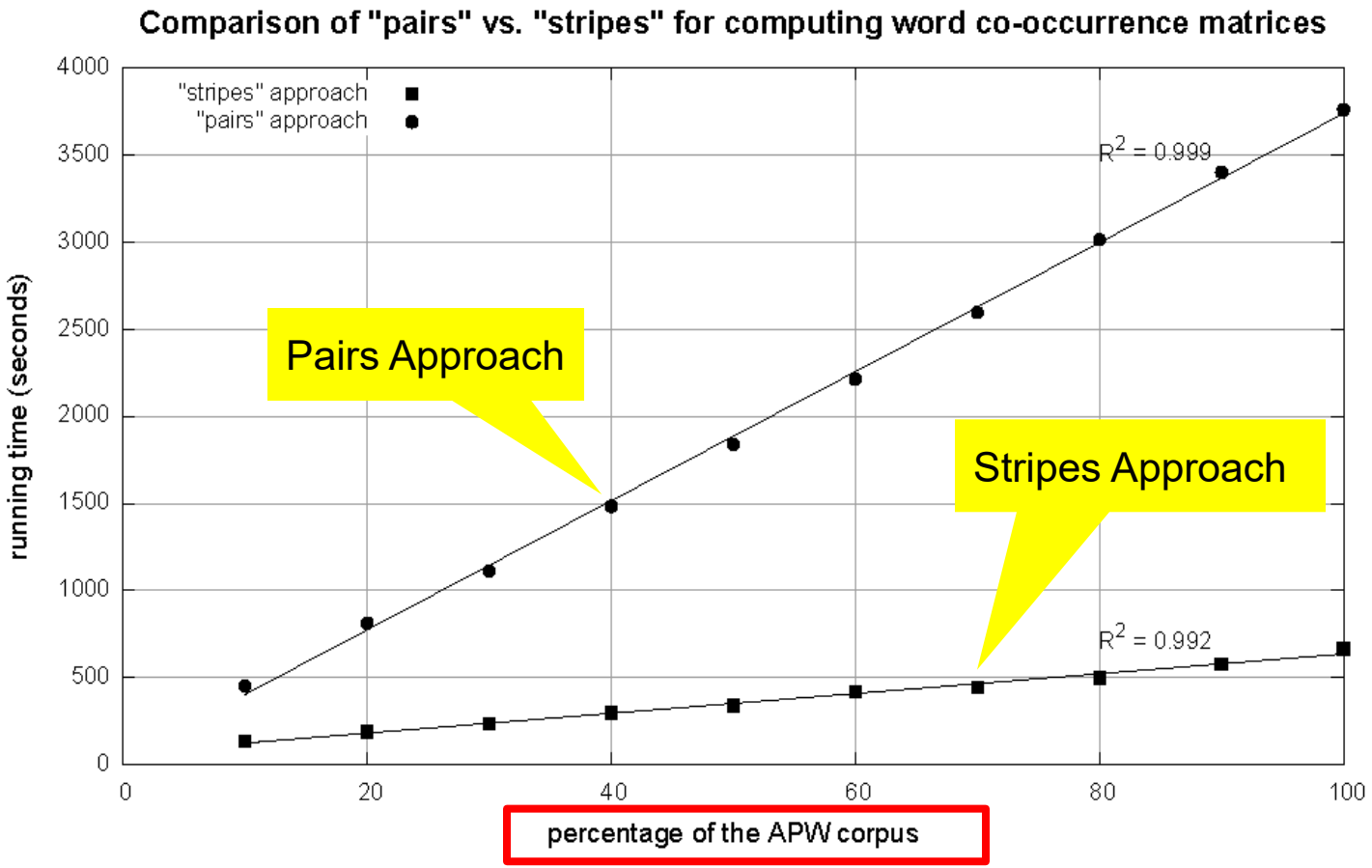
Input: key w, strips $[H_1, H_2, \ldots]$
$H_f \leftarrow$ a new list
for each strip $H \in [H_1, H_2, \ldots]$ do
  $H_f \leftarrow H_f + H$
emit(key w, strip H_f)

Consider how to implement it ☺

- By line
- Preserve state across lines

Test Results Stripes vs Pairs

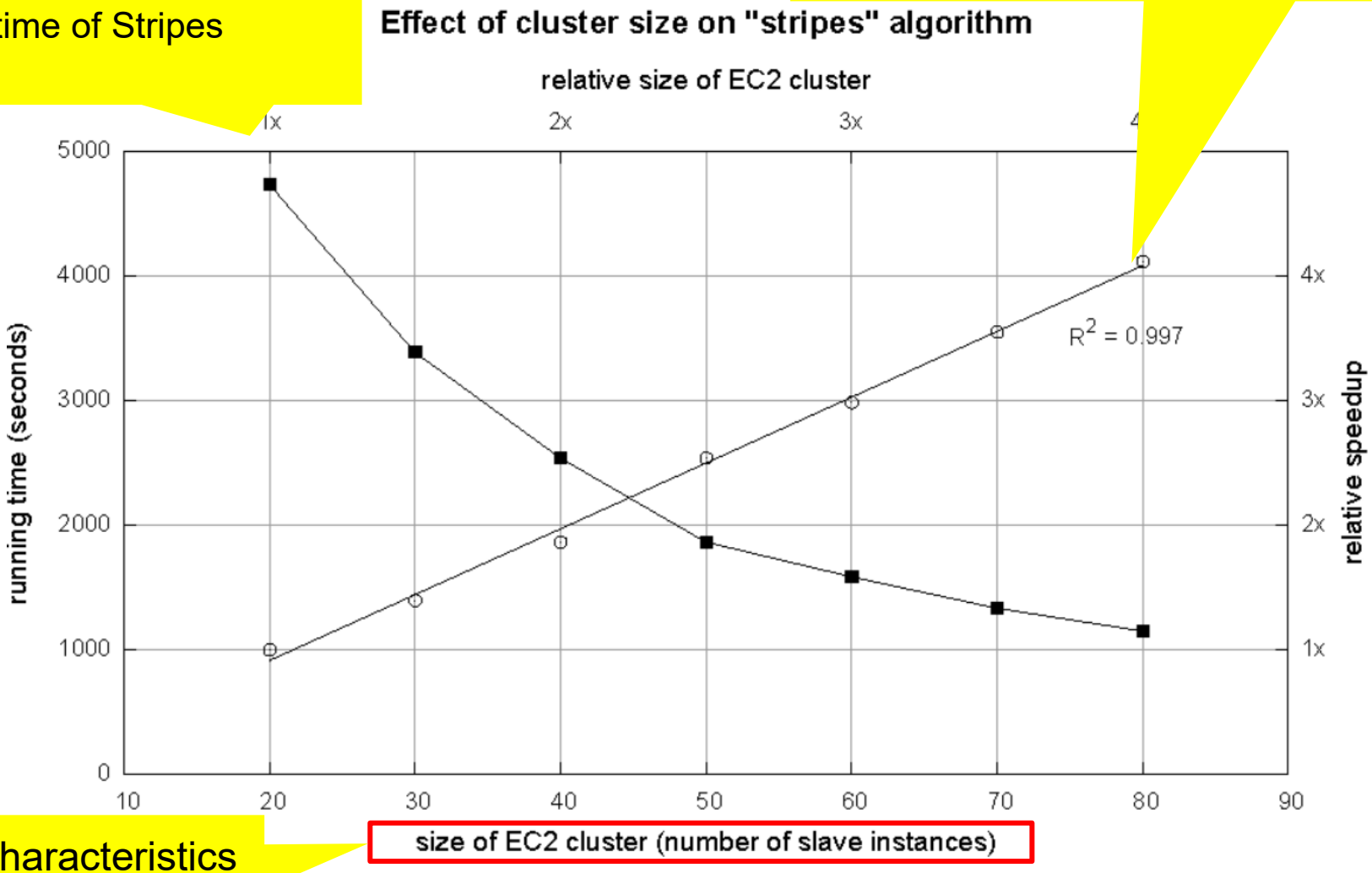
Cluster size: 38 slave instances (nodes)
Data Source:
Associated Press Worldstream (APW) of the English Gigaword Corpus (v3), which contains 2.27 million documents (1.8 GB compressed, 5.7 GB uncompressed)



More Test Results - Stripes

Running time speed up by the number of times – Stripes approach

Running time of Stripes approach



Scaling characteristics in terms of increasing Hadoop cluster size

End of Part 2

Part 3

Co-occurrence
Relative Frequencies

Answer Question 2 : According to co-occurrence matrix,

- Is "*blue*" an important word to "*cat*"?
- Is "*blue*" an important word to "*boat*"?

	purple	pink	Yellow	blue	red	black
knife	51	20	84	0	3	0
cat	52	58	4	4	6	500
dog	115	89	10	42	33	17
boat	0	0	0	4	0	0
cup	98	14	6	2	1	0
pig	12	17	3	2	9	27
berry	11	2	2	0	18	0

4/624

Answer Question 2 : According to co-occurrence matrix,

- Is "*blue*" an important word to "*cat*"?
- Is "*blue*" an important word to "*boat*"?

	purple	pink	Yellow	blue	red	black
knife	51	20	84	0	3	0
cat	52	58	4	4	6	500
dog	115	89	10	42	33	17
boat	0	0	0	4	0	0
cup	98	14	6	2	1	0
pig	12	17	3	2	9	27
berry	11	2	2	0	18	0

4/624

4/4

Relative Frequencies

- How do we estimate relative frequencies from counts?

$$f(B|A) = \frac{N(A,B)}{N(A)} = \frac{N(A,B)}{\sum_{B'} N(A,B')}$$

Relative Frequencies

- How do we estimate relative frequencies from counts?

$$f(B|A) = \frac{N(A,B)}{N(A)} = \frac{N(A,B)}{\sum_{B'} N(A,B')}$$

	purple	pink	Yellow	blue	red	black
cat	52	58	4	4	6	500

624

Relative Frequencies

- How do we estimate relative frequencies from counts?

$$f(B|A) = \frac{N(A,B)}{N(A)} = \frac{N(A,B)}{\sum_{B'} N(A,B')}$$

f(blue|cat)= 4/624

	purple	pink	Yellow	blue	red	black
cat	52	58	4	4	6	500

4/624

N(A, B) is known

$$f(B|A) = \frac{N(A,B)}{N(A)} = \frac{N(A,B)}{\sum_{B'} N(A,B')}$$

How to compute summation of N(A, B) for all B?

	purple	pink	Yellow	blue	red	black
cat	52	58	4	4	6	500

624?

$$f(B|A) = \frac{N(A,B)}{N(A)} = \frac{N(A,B)}{\sum_{B'} N(A,B')}$$

How to compute summation of N(A, B) for all B?

Two MapReduce Algorithms

- Stripes Approach
- Pairs Approach

Relative Frequencies - Stripes

("hadoop", {{"apache", 1}, {"consists", 1}, {"core", 1}, {"storage" 1}, {"known", 1}, {"distributed", 1}, {"file", 2}, {"part" 1}, {"model", 1}, {"programming", 1}, {"splits" 1} })

$A \rightarrow \{B_1:3, B_2:12, B_3:7, B_4:1, \ldots\}$

Easy!

- One pass to compute sum(A, *)=3+12+7+1+…
- One pass to compute N(A,B) for each B, and then compute f(B|A) = N(A,B)/sum(A,*)

$f(B_2|A)=12/sum(A,*)$

$$f(B|A) = \frac{N(A,B)}{N(A)} = \frac{N(A,B)}{\sum_{B'} N(A,B')}$$

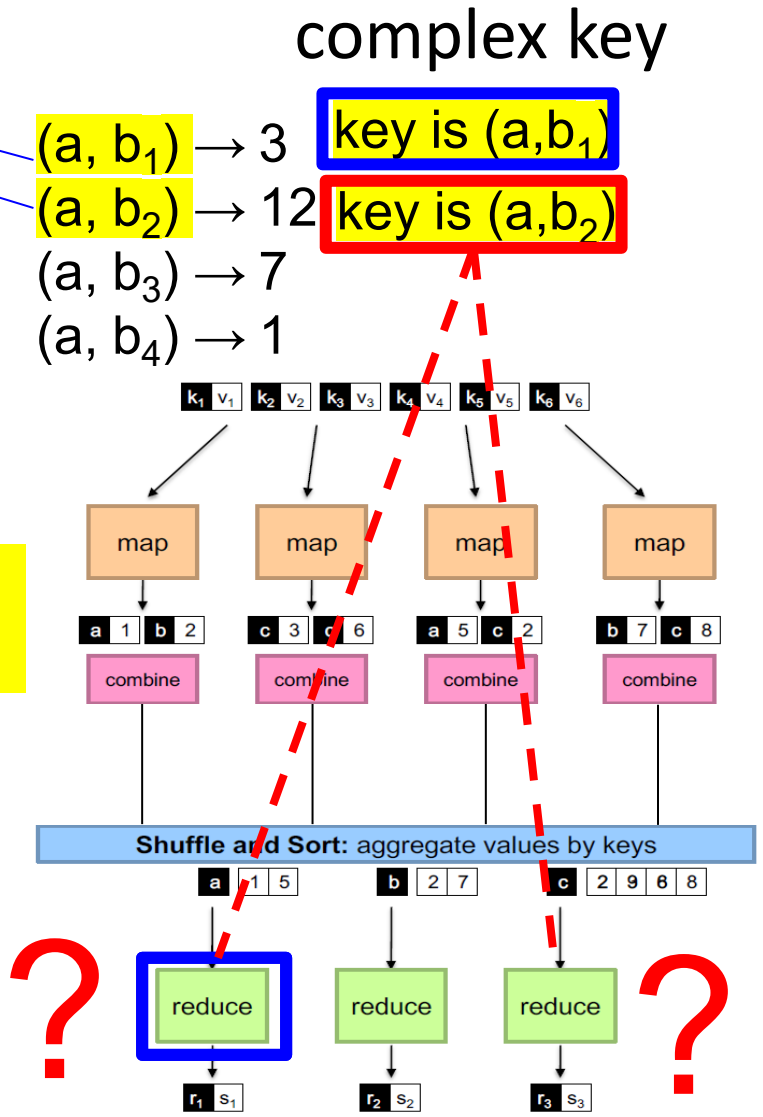
Two MapReduce Algorithms

- Stripes Approach
- Pairs Approach

Relative Frequencies - Pairs

((“Hadoop”, “apache”), 1)
((“Hadoop”, “consists”), 1)
((“Hadoop”, “core”), 1)
((“Hadoop”, “storage”), 1)
((“Hadoop”, “known”), 1)
((“Hadoop”, “distributed”), 1)
((“Hadoop”, “file”), 2)
((“Hadoop”, “part”), 1)
((“Hadoop”, “model”), 1)
((“Hadoop”, “programming”), 1)
((“Hadoop”, “splits”), 1)

No guarantee that (a, b1) and (a, b2) are assigned to the same reducer.

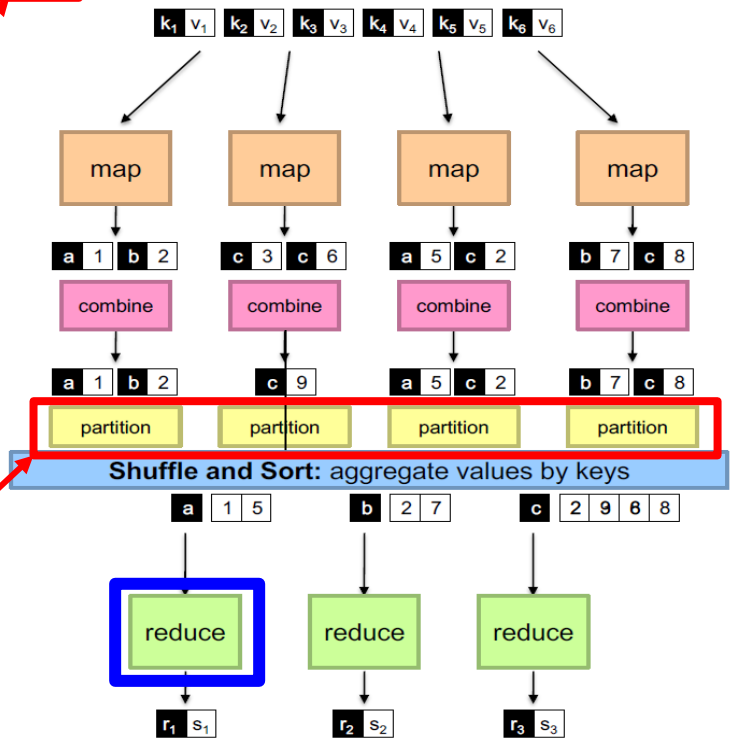


Relative Frequencies - Pairs

((“Hadoop”, “apache”), 1)
((“Hadoop”, “consists”), 1)
((“Hadoop”, “core”), 1)
((“Hadoop”, “storage”), 1)
((“Hadoop”, “known”), 1)
((“Hadoop”, “distributed”), 1)
((“Hadoop”, “file”), 2)
((“Hadoop”, “part”), 1)
((“Hadoop”, “model”), 1)
((“Hadoop”, “programming”), 1)
((“Hadoop”, “splits”), 1)

complex key

$(a, b_1) \rightarrow 3$ Partition on “a” only
$(a, b_2) \rightarrow 12$ Partition on “a” only
$(a, b_3) \rightarrow 7$
$(a, b_4) \rightarrow 1$



No guarantee that (a, b1) and (a, b2) are assigned to the same reducer.

To produce the desired behaviour, we must design a custom partitioner that only pays attention to the **left word**.

- That is, the partitioner should partition based on the hash of the left word only.

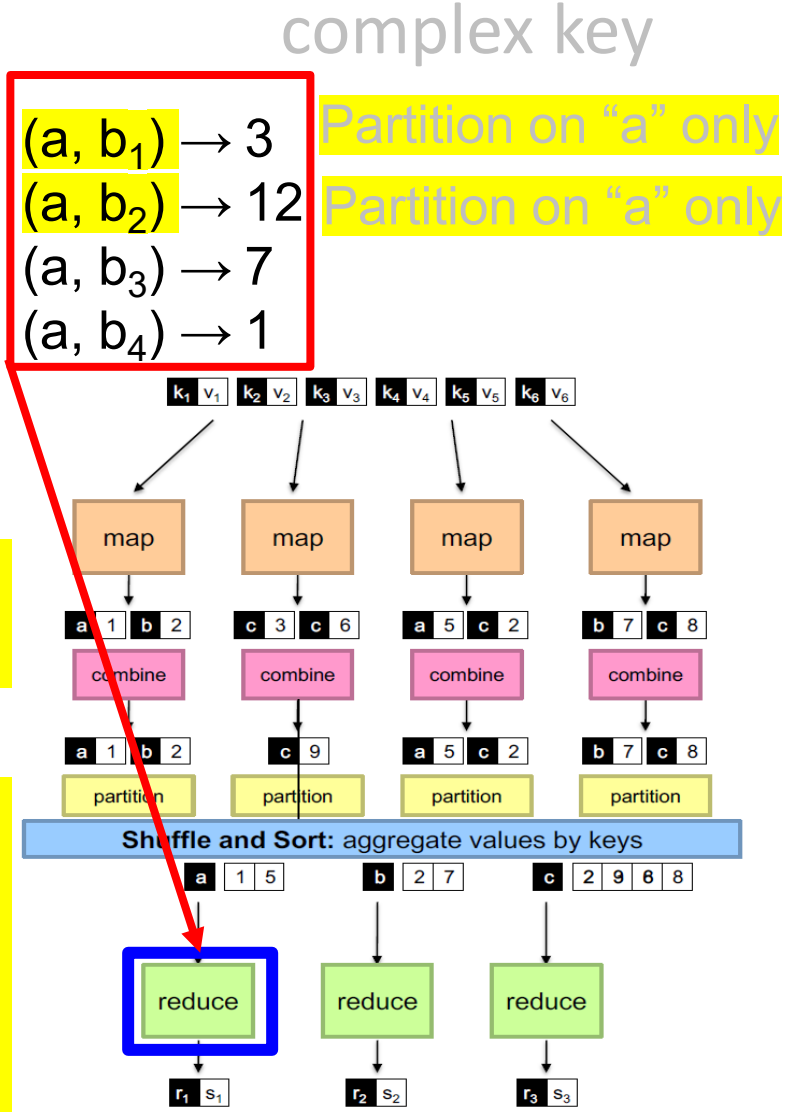
Relative Frequencies - Pairs

(("Hadoop", "apache"), 1)
(("Hadoop", "consists"), 1)
(("Hadoop", "core"), 1)
(("Hadoop", "storage"), 1)
(("Hadoop", "known"), 1)
(("Hadoop", "distributed"), 1)
(("Hadoop", "file"), 2)
(("Hadoop", "part"), 1)
(("Hadoop", "model"), 1)
(("Hadoop", "programming"), 1)
(("Hadoop", "splits"), 1)

No guarantee that (a, b1) and (a, b2) are assigned to the same reducer.

To produce the desired behaviour, we must design a custom partitioner that only pays attention to the **left word**.

- That is, the partitioner should partition based on the hash of the left word only.



Relative Frequencies - Pairs

(("Hadoop", "apache"), 1)
(("Hadoop", "consists"), 1)
(("Hadoop", "core"), 1)
(("Hadoop", "storage"), 1)
(("Hadoop", "known"), 1)
(("Hadoop", "distributed"), 1)
(("Hadoop", "file"), 2)
(("Hadoop", "part"), 1)
(("Hadoop", "model"), 1)
(("Hadoop", "programming"), 1)
(("Hadoop", "splits"), 1)

complex key

$(a, b_1) \rightarrow 3$ Partition on "a" only

$(a, b_2) \rightarrow 12$ Partition on "a" only

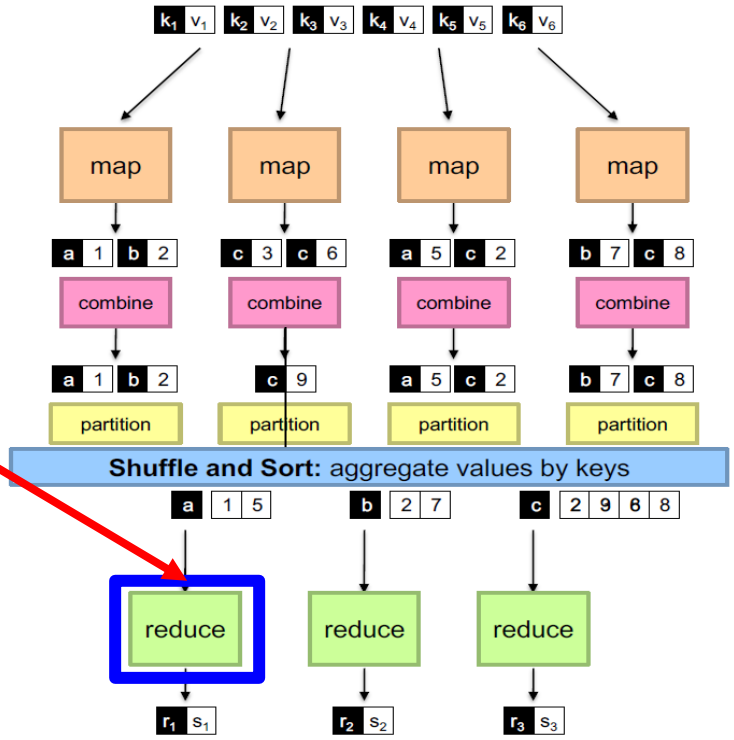
$(a, b_3) \rightarrow 7$

$(a, b_4) \rightarrow 1$

No guarantee that (a, b1) and (a, b2) are assigned to the same reducer.

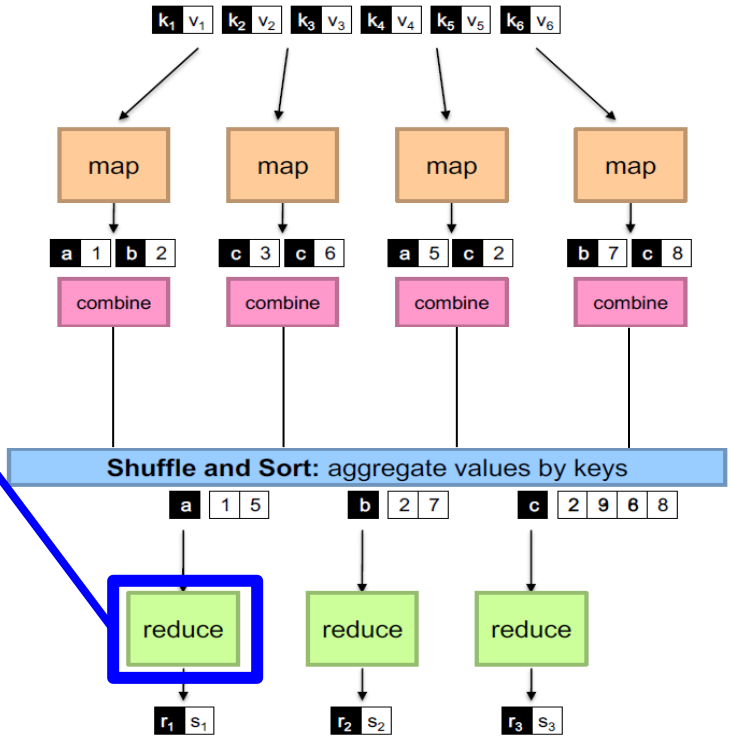
To produce the desired behaviour, we must design a custom partitioner that only pays attention to the **left word**.

- That is, the partitioner should partition based on the hash of the left word only.



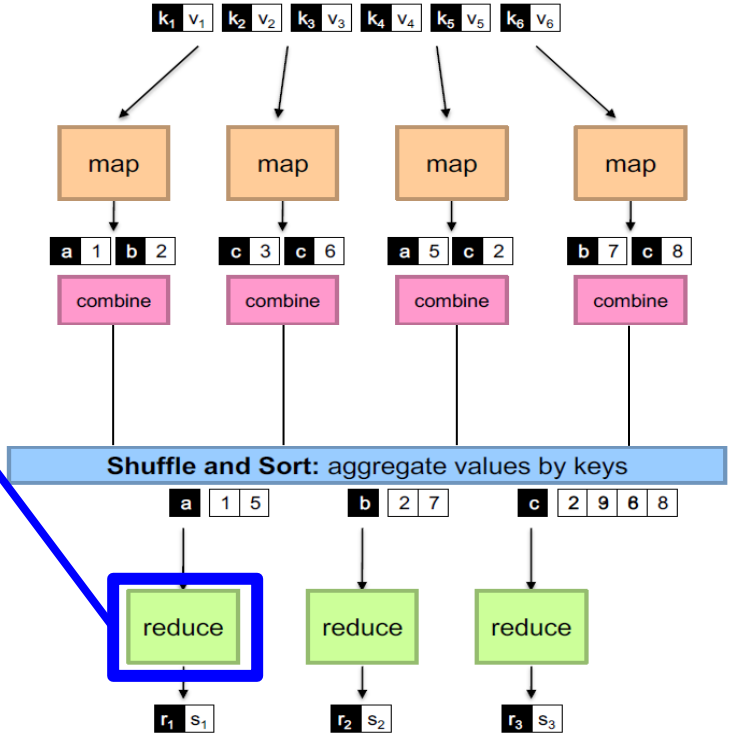
key	values	
(dog, aardvark)	[2,1]	3
(dog, aardwolf)	[1]	1
...		
(dog, zebra)	[2,1,1,1]	5

After partition, shuffle, sort



key	values	
$(\text{dog}, *)$	[6327, 8514, . . .]	compute marginal: $\sum_{w'} N(\text{dog}, w') = 42908$
(dog, aardvark)	[2,1]	$f(\text{aardvark}\|\text{dog}) = 3/42908$
(dog, aardwolf)	[1]	$f(\text{aardwolf}\|\text{dog}) = 1/42908$
. . .		
(dog, zebra)	[2,1,1,1]	$f(\text{zebra}\|\text{dog}) = 5/42908$

After partition, shuffle, sort



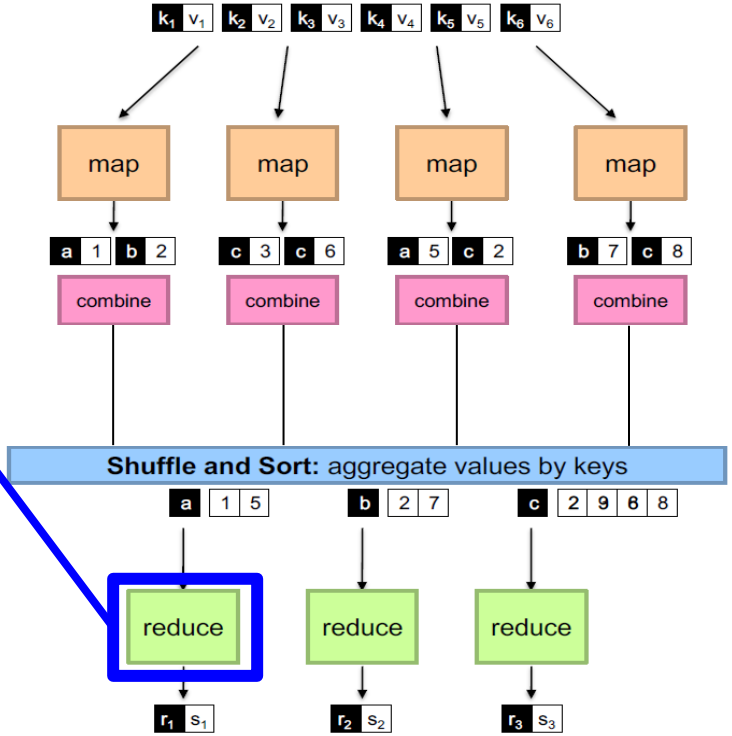
key	values	
$(\text{dog}, *)$	$[6327, 8514, \ldots]$	compute marginal: $\sum_{w'} N(\text{dog}, w') = 42908$

How to obtain (dog, *)



Two solutions

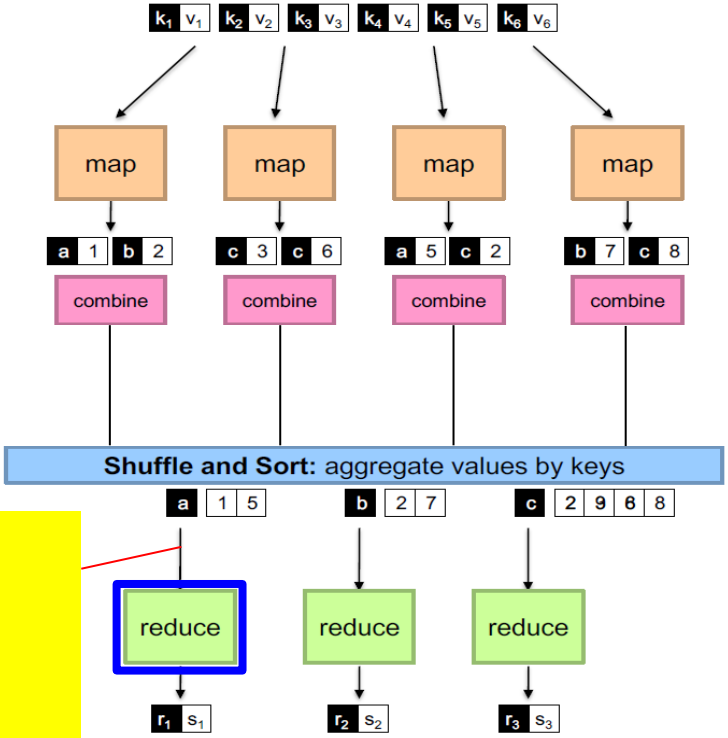
After partition, shuffle, sort



Memory Requirement may be huge

in memory

(dog, aardvark) [2,1]
(dog, aardwolf) [1]
…
(dog, zebra) [2,1,1,1]



key	values	
(dog, ∗)	[6327, 8514, . . .]	compute marginal: $\sum_{w'} N(\text{dog}, w') = 42908$
(dog, aardvark)	[2,1]	$f(\text{aardvark}\|\text{dog}) = 3/42908$
(dog, aardwolf)	[1]	$f(\text{aardwolf}\|\text{dog}) = 1/42908$
. . .		
(dog, zebra)	[2,1,1,1]	$f(\text{zebra}\|\text{dog}) = 5/42908$

First Solution

- To calculate (dog, *), keep all (dog, xxx) [xx, xx] in memory
- Calculate relative frequency for each (dog, xxx)

key	values	
$(\text{dog}, *)$	$[6327, 8514, \ldots]$	compute marginal: $\sum_{w'} N(\text{dog}, w') = 42908$

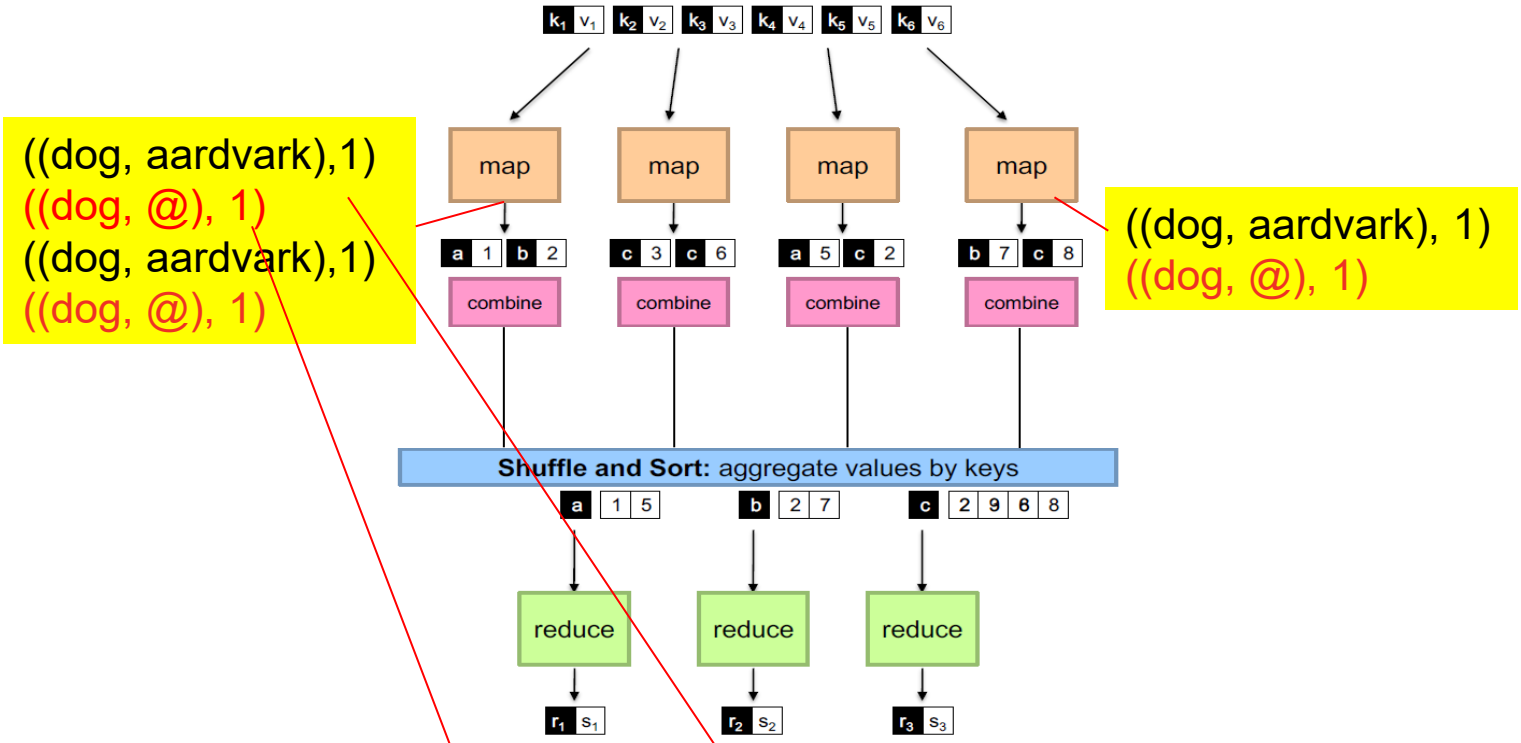
How to obtain (dog, *)



Two solutions

Second Solution

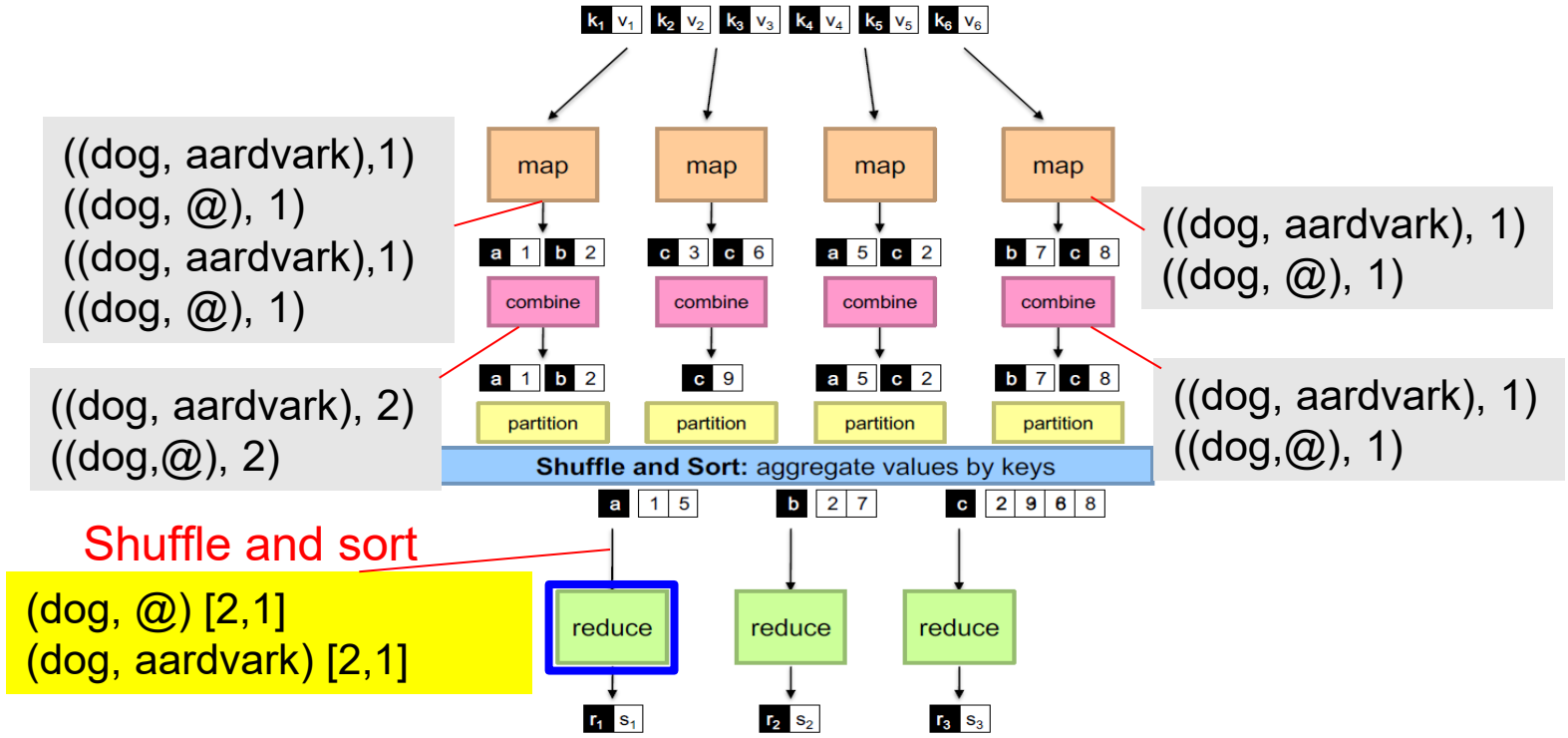
- mapper additionally emits one (dog,@) for each (dog, xxxx)
- arrange (dog,@) be the first after sort, then aggregate in sorted order



key	values	
(dog, aardvark)	[2,1]	$f(\text{aardvark}\|\text{dog}) = 3/42908$
(dog, aardwolf)	[1]	$f(\text{aardwolf}\|\text{dog}) = 1/42908$
...		
(dog, zebra)	[2,1,1,1]	$f(\text{zebra}\|\text{dog}) = 5/42908$

Second Solution

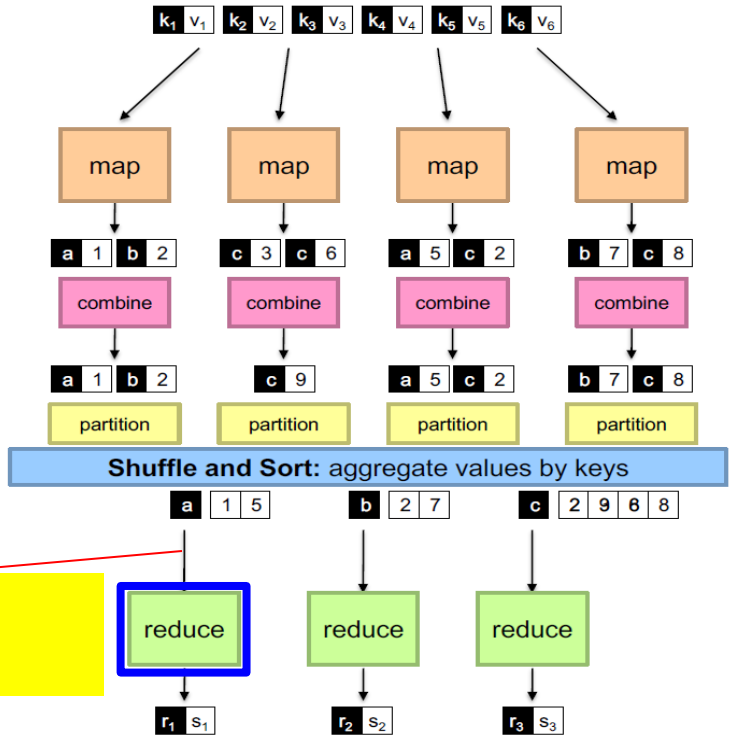
- mapper additionally emits one (dog,@) for each (dog, xxxx)
- arrange (dog,@) be the first after sort, then aggregate in sorted order



key	values	
(dog, aardvark)	[2,1]	$f(\text{aardvark}\|\text{dog}) = 3/42908$
(dog, aardwolf)	[1]	$f(\text{aardwolf}\|\text{dog}) = 1/42908$
...		
(dog, zebra)	[2,1,1,1]	$f(\text{zebra}\|\text{dog}) = 5/42908$

Second Solution

- mapper additionally emits one (dog,@) for each (dog, xxxx)
- arrange (dog,@) be the first after sort, then aggregate in sorted order



Shuffle and sort

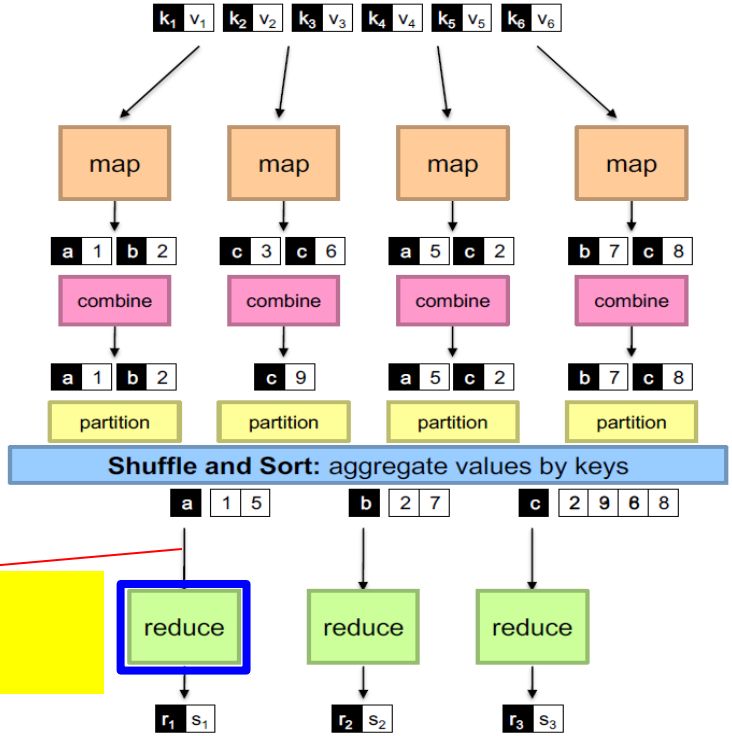
(dog, @) [2,1]
(dog, aardvark) [2,1]

First in order

key	values	
(dog, ∗)	[6327, 8514, . . .]	compute marginal: $\sum_{w'} N(\text{dog}, w') = 42908$
(dog, aardvark)	[2,1]	$f(\text{aardvark}\|\text{dog}) = 3/42908$
(dog, aardwolf)	[1]	$f(\text{aardwolf}\|\text{dog}) = 1/42908$
. . .		
(dog, zebra)	[2,1,1,1]	$f(\text{zebra}\|\text{dog}) = 5/42908$

Second Solution

- mapper additionally emits one (dog,*) for each (dog, xxxx)
- arrange (dog,*) be the first after sort, then aggregate in sorted order

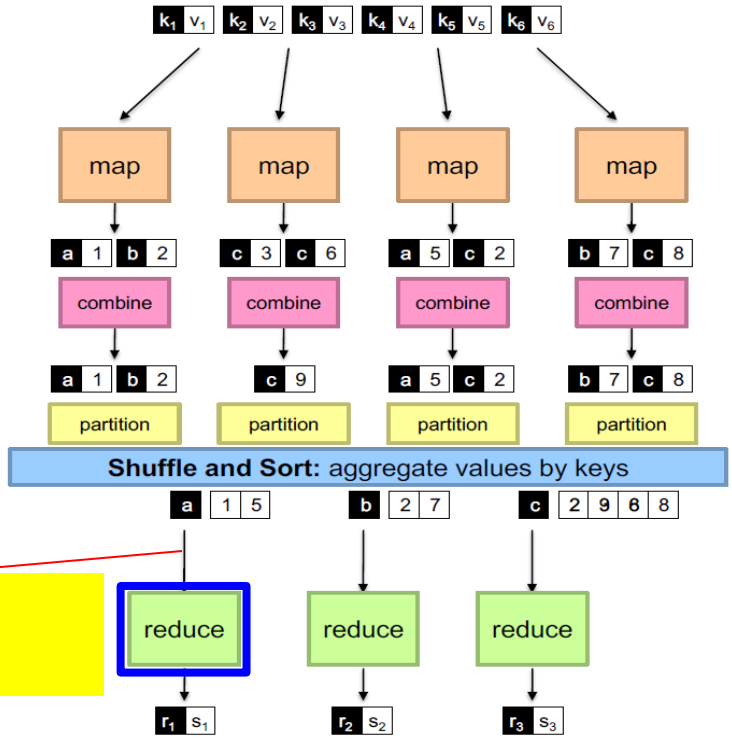


(dog, *) [2,1]
(dog, aardvark) [2,1]

key	values	
~~(dog, *)~~	~~[6327, 8514, ...]~~	~~compute marginal: $\sum_{w'} N(\text{dog}, w') = 42908$~~
(dog, aardvark)	[2,1]	$f(\text{aardvark}\|\text{dog}) = 3/42908$
(dog, aardwolf)	[1]	$f(\text{aardwolf}\|\text{dog}) = 1/42908$
...		
(dog, zebra)	[2,1,1,1]	$f(\text{zebra}\|\text{dog}) = 5/42908$

Second Solution

- mapper additionally emits one (dog,*) for each (dog, xxxx)
- arrange (dog,*) be the first after sort, then aggregate in sorted order



(dog, *) [2,1]
(dog, aardvark) [2,1]

key	values	
~~(dog, *)~~	~~[6327, 8514, ...]~~	~~compute marginal: $\sum_{w'} N(\text{dog}, w') = 42908$~~
~~(dog, aardvark)~~	~~[2,1]~~	~~$f(\text{aardvark}\|\text{dog}) = 3/42908$~~
(dog, aardwolf)	[1]	$f(\text{aardwolf}\|\text{dog}) = 1/42908$
...		
(dog, zebra)	[2,1,1,1]	$f(\text{zebra}\|\text{dog}) = 5/42908$

Second Solution

- mapper additionally emits one (dog,*) for each (dog, xxxx)
- arrange (dog,*) be the first after sort, then aggregate in sorted order

This design pattern is called “order inversion“

- It is so named because through proper coordination, we can access the result of a computation in the reducer (for example, an aggregate statistic) before processing the data needed for that computation.
- The key insight is to convert the sequencing of computations into a sorting problem.

Second Solution

- mapper additionally emits one (dog,*) for each (dog, xxxx)
- arrange (dog,*) be the first after sort, then aggregate in sorted order

This design pattern is called “order inversion“

- It is so named because through proper coordination, we can access the result of a computation in the reducer (for example, an aggregate statistic) before processing the data needed for that computation.
- The key insight is to convert the sequencing of computations into a sorting problem.

- In most cases, an algorithm requires data in some fixed order: by controlling how keys are sorted and how the key space is partitioned, we can present data to the reducer in the order necessary to perform the proper computations.

Second Solution

- mapper additionally emits one (dog,*) for each (dog, xxxx)
- arrange (dog,*) be the first after sort, then aggregate in sorted order

This design pattern is called “order inversion“

- It is so named because through proper coordination, we can access the result of a computation in the reducer (for example, an aggregate statistic) before processing the data needed for that computation.
- The key insight is to convert the sequencing of computations into a sorting problem.

- In most cases, an algorithm requires data in some fixed order: by controlling how keys are sorted and how the key space is partitioned, we can present data to the reducer in the order necessary to perform the proper computations.
- This greatly cuts down on the amount of partial results that the reducer needs to hold in memory.

Second Solution

- mapper additionally emits one (dog,*) for each (dog, xxxx)
- arrange (dog,*) be the first after sort, then aggregate in sorted order

End of Part 3

Next Week
MapReduce Clustering and Programming